

Lab 3 — Exercise: Wrangling the Google Play Store Dataset

ITCS 3162 — Introduction to Data Mining

Name: Daniel Potapov **Date:** 05/28/2026

This dataset is *messier* than diamonds — it's scraped from the Google Play Store and has the kinds of problems you'll see in the wild: numbers stored as strings, units mixed with values, duplicates, weird placeholder strings, and missing values. That's the point: you'll do real cleaning.

Source: Kaggle's "Google Play Store Apps" dataset (mirrored on GitHub for stable URL access).

When you're done, **Restart & Run All**, download as `.ipynb`, and submit via Canvas.

Setup

Run this cell. It loads the dataset directly from a GitHub URL.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

URL = "https://raw.githubusercontent.com/sumitgirwal/google-play-store-data-analysis/master/googleplaystore.csv"
df = pd.read_csv(URL)
df.head()
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0 and above
	Coloring									Art & Design	January 7, 2018	1.0.0	4.0 and above

Next steps: [Generate code with df](#) [New interactive sheet](#)

Exercise 1 — Initial inspection (10 pts)

In the cell below, print **all three**:

1. The shape of the DataFrame (rows, columns)
2. The output of `df.info()`
3. The output of `df.describe(include="all")`

Then in the markdown cell, answer the questions.

```
# TODO: print shape, info(), describe(include='all')
print(df.shape)
print(df.info())
print(df.describe(include="all"))
```

```
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   App                  10841 non-null  object
1   Category              10841 non-null  object
2   Rating                9367 non-null   float64
3   Reviews               10841 non-null  object
4   Size                  10841 non-null  object
5   Installs              10841 non-null  object
6   Type                  10840 non-null  object
7   Price                 10841 non-null  object
8   Content Rating        10840 non-null  object
```

```

11 Current Ver      10833 non-null object
12 Android Ver      10838 non-null object
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
None

```

	App	Category	Rating	Reviews	Size	Installs
count	10841	10841	9367.000000	10841	10841	10841
unique	9660	34	NaN	6002	462	22
top	ROBLOX	FAMILY	NaN	0	Varies with device	1,000,000+
freq	9	1972	NaN	596	1695	1579
mean	NaN	NaN	4.193338	NaN	NaN	NaN
std	NaN	NaN	0.537431	NaN	NaN	NaN
min	NaN	NaN	1.000000	NaN	NaN	NaN
25%	NaN	NaN	4.000000	NaN	NaN	NaN
50%	NaN	NaN	4.300000	NaN	NaN	NaN
75%	NaN	NaN	4.500000	NaN	NaN	NaN
max	NaN	NaN	19.000000	NaN	NaN	NaN

	Type	Price	Content	Rating	Genres	Last Updated
count	10840	10841	10840	10841	10841	10841
unique	3	93	6	120	1378	
top	Free	0	Everyone	Tools	August 3, 2018	
freq	10039	10040	8714	842	326	
mean	NaN	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	NaN	

	Current Ver	Android Ver
count	10833	10838
unique	2832	33
top	Varies with device	4.1 and up
freq	1459	2451
mean	NaN	NaN
std	NaN	NaN
min	NaN	NaN
25%	NaN	NaN
50%	NaN	NaN
75%	NaN	NaN
max	NaN	NaN

Answer the following:

1. How many rows and columns does the dataset have?
2. Look at the `Reviews`, `Size`, `Installs`, and `Price` columns. What dtype does pandas assign them? Why is that a problem for a numeric column like `Reviews`?
3. Which columns appear to have missing values?

YOUR ANSWERS:

1. Columns = 10841 and Rows = 13
2. Pandas assigns the them the object dtype. This is a problem mainly because the object dtype is a generic python object, which doesn't allow us to use mathematical functions like mean or std on it, since the generic python object is treated not as a number thus cannot be used in mathematical python functions.
3. Every single column has missing values

Exercise 2 — Missing values (10 pts)

In the cell below, compute (a) the count of missing values per column, sorted descending, and (b) the **percentage** of missing values per column rounded to 2 decimal places. Then drop any row that is missing the `Rating` column (since it's the most important missingness to handle) and store the result in `df_r`.

```

# TODO: counts of missing values per column (sorted descending)
missing_counts = df.isnull().sum().sort_values(ascending=False)

# TODO: percentage missing per column (rounded to 2 decimal places)
missing_percentages = ((df.isnull().sum() / len(df)) * 100).round(2)

# TODO: create df_r by dropping rows where Rating is NaN; print its new shape
df_r = df.dropna(subset=['Rating'])

```

Exercise 3 — Duplicates (10 pts)

There are duplicate rows in this dataset (the same app appears multiple times). In the cell below:

1. Print how many fully-duplicate rows exist in `df_r`.
2. How many duplicate values are there in the `App` column specifically (apps appearing under different categories)?
3. Create `df_dedup` by dropping fully-duplicate rows and print its new shape.

```
# TODO: fully duplicate rows
fully_duplicates = df_r.duplicated().sum()
print(fully_duplicates)

# TODO: duplicate App names (use df_r["App"].duplicated().sum())
app_duplicates = df_r["App"].duplicated().sum()
print(app_duplicates)

# TODO: df_dedup = df_r.drop_duplicates(); print shape
df_dedup = df_r.drop_duplicates()
print(df_dedup.shape)
```

```
474
1170
(8893, 13)
```

Exercise 4 — Cleaning the `Installs` column (15 pts)

The `Installs` column looks like `"1,000,000+"` — a string with a `+` and commas. Convert it to an integer column. Steps:

1. Inspect a few unique values: `df_dedup["Installs"].unique()[:10]`
2. Remove `+` and `,` characters with `.str.replace()`
3. Convert to int with `.astype(int)` (or use `pd.to_numeric` with `errors="coerce"` if any values look weird)
4. Assign the cleaned values back to `df_dedup["Installs"]`
5. Verify with `df_dedup["Installs"].dtype` and `.head()`

```
# TODO: inspect unique values
print("First 10 unique values:", df_dedup["Installs"].unique()[:10])

# TODO: clean Installs to integer
df_dedup = df_r.drop_duplicates().copy()

df_dedup["Installs"] = df_dedup["Installs"].str.replace("+", "", regex=False).str.replace(",", "", regex=False)
df_dedup["Installs"] = pd.to_numeric(df_dedup["Installs"], errors='coerce')
df_dedup = df_dedup.dropna(subset=["Installs"])
df_dedup["Installs"] = df_dedup["Installs"].astype(int)

print(df_dedup["Installs"].dtype)
print(df_dedup["Installs"].head())
```

```
First 10 unique values: ['10,000+' '500,000+' '5,000,000+' '50,000,000+' '100,000+' '50,000+'
'1,000,000+' '10,000,000+' '5,000+' '100,000,000+']
int64
0      10000
1     500000
2    5000000
3   50000000
4     100000
Name: Installs, dtype: int64
```

Exercise 5 — Cleaning the `Price` column (15 pts)

`Price` looks like `"0"` for free apps and `"$4.99"` for paid ones. Convert it to a float.

1. Use `.str.replace("$", "", regex=False)` then `.astype(float)`.
2. Watch for any unexpected values (the real dataset has at least one weird row — the column may have a non-numeric placeholder that `pd.to_numeric(..., errors="coerce")` handles gracefully).
3. After cleaning, print how many paid apps there are (where `Price > 0`).

```
# TODO: clean Price to float
df_dedup["Price"] = df_dedup["Price"].str.replace("$", "", regex=False)
df_dedup["Price"] = pd.to_numeric(df_dedup["Price"], errors='coerce')
df_dedup = df_dedup.dropna(subset=["Price"])
df_dedup["Price"] = df_dedup["Price"].astype(float)

# TODO: count paid apps
paid_apps_count = (df_dedup["Price"] > 0).sum()
print(paid_apps_count)
```

613

Exercise 6 — Filtering and a derived column (15 pts)

1. Create `top_apps`, containing only apps with `Rating >= 4.5` and `Installs >= 1_000_000`.
2. Print its shape and the top 5 categories of those apps by count.
3. In `df_dedup`, add a new column `Revenue` defined as `Price * Installs` (zero for free apps).
4. Show the top 10 apps by `Revenue`.

```
# TODO: top_apps and category counts
top_apps = df_dedup[(df_dedup["Rating"] >= 4.5) & (df_dedup["Installs"] >= 1_000_000)]
print("Shape: ", top_apps.shape)
print(top_apps["Category"].value_counts().head(5))

# TODO: Revenue column and top 10
df_dedup["Revenue"] = df_dedup["Price"] * df_dedup["Installs"]
top_10_revenue = df_dedup.sort_values(by="Revenue", ascending=False)[["App", "Category", "Price", "Installs", "Revenue"]].head()
print(top_10_revenue)
```

```
Shape: (1234, 13)
Category
GAME                272
FAMILY               171
TOOLS                89
HEALTH_AND_FITNESS  82
PRODUCTIVITY        62
Name: count, dtype: int64
```

	App	Category	Price	Installs	Revenue
4347	Minecraft	FAMILY	6.99	10000000	69900000.0
2241	Minecraft	FAMILY	6.99	10000000	69900000.0
5351	I am rich	LIFESTYLE	399.99	100000	39999000.0
5356	I Am Rich Premium	FINANCE	399.99	50000	19999500.0
4034	Hitman Sniper	GAME	0.99	10000000	9900000.0
7417	Grand Theft Auto: San Andreas	GAME	6.99	1000000	6990000.0
2883	Facetune - For Free	PHOTOGRAPHY	5.99	1000000	5990000.0
5578	Sleep as Android Unlock	LIFESTYLE	5.99	1000000	5990000.0
8804	DraStic DS Emulator	GAME	4.99	1000000	4990000.0
4367	I'm Rich - Trump Edition	LIFESTYLE	400.00	10000	4000000.0

Exercise 7 — Group-by summary (15 pts)

Using `df_dedup`, produce a single summary table with one row per **Category**, containing:

- `n_apps` — number of apps in that category
- `mean_rating` — average rating (rounded to 2 decimals)
- `mean_installs` — average installs (rounded to 0 decimals)
- `total_revenue` — sum of `Revenue`

Sort by `total_revenue` descending. Show only the top 10 rows.

```
# TODO: groupby summary
summary = df_dedup.groupby("Category").agg(
    n_apps=("App", "count"),
    mean_rating=("Rating", lambda x: round(x.mean(), 2)),
    mean_installs=("Installs", lambda x: round(x.mean(), 0)),
    total_revenue=("Revenue", "sum")
)

top = summary.sort_values(by="total_revenue", ascending=False).head(10)
print(top)
```

	n_apps	mean_rating	mean_installs	total_revenue
Category				
FAMILY	1718	4.19	5844663.0	1.857743e+08
LIFESTYLE	305	4.10	1753250.0	5.758394e+07
GAME	1074	4.28	29370449.0	4.098684e+07
FINANCE	317	4.13	2430008.0	2.572664e+07
PHOTOGRAPHY	304	4.18	31977773.0	8.941050e+06
MEDICAL	302	4.18	139612.0	8.371355e+06
PERSONALIZATION	310	4.33	6691461.0	7.786310e+06
TOOLS	734	4.05	15600442.0	5.462910e+06
SPORTS	286	4.23	5344516.0	4.706154e+06
PRODUCTIVITY	334	4.20	37314581.0	4.304452e+06

Exercise 8 — Reflection (10 pts)

In 4–6 sentences, answer all of:

1. Which cleaning step would have been impossible to skip if you wanted to compute average install counts? Why?
2. What's one piece of information in the raw data that you *could* extract with more work but didn't (e.g., the `Size` column, the `Last Updated` column)? How would you approach it?
3. Did anything in the data surprise you?

YOUR ANSWER:

1. Step number 4, cleaning installs portion. This is a key part in the cleaning process because of the way the installs data is stored. It uses commas or plus signs in some of the data entries, thus doing any sort of math operation, in this case averaging the installs, not possible, since the data type is a string .
2. I could choose to work more with the size column in the data. The main steps I would need to take to clean this column up, is removing or replacing any null/missing values, and making sure that every data point in the column is an integer type and not a string, because some of the data entries have strings like "Varies with device" which we can obviously not use, similar to the installs portion.
3. Main thing that surprised me, is how much installs apps actually get.

Submission checklist

- ☐ Name and date filled in
- ☐ All TODO cells completed and run
- ☐ All `YOUR ANSWER` prompts replaced
- ☐ **Restart & Run All** completes without errors
- ☐ Downloaded as `.ipynb` and uploaded to Canvas